01 · Log vitals and the anomaly model

A citizen enters their measurements; the backend checks them, saves them, scores them and explains the result.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

On the **Log vitals** page a citizen fills in whatever they measured (blood pressure, heart rate, temperature, SpO2, breathing rate, glucose, weight) and optional symptoms. After saving, they see whether the reading was flagged, why, and a table of how each vital was checked.

Behind the scenes the reading is stored, scored by the anomaly model, and, if flagged, an alert is created and the flag appears in the health worker's worklist.

Files involved

Only the files this feature uses.

```
backend/app/
├── services/metrics.py        the list of vitals: name, unit, normal range, plausible range
├── api/routes/vitals.py       GET /health/metrics, POST /health/vitals
├── services/readings.py       record_reading(): save + score + alert
├── services/scoring.py        score_reading(): the anomaly model
├── services/triage.py         score -> severity; features -> a plain-language sentence
├── models/health_record.py    table health_records (one reading)
├── models/processed_data.py   table processed_data (its score)
└── schemas/health.py          VitalsIn, VitalsResult, VitalCheck
frontend/src/
├── pages/citizen/SubmitVitals.tsx       the form and the result screen
├── services/health.ts                   getMetrics(), submitVitals()
└── components/app/ModelInsights.tsx     ScoreBreakdown: "How this reading was checked"
```

How the pieces connect

1. The page calls `getMetrics()` to learn which vitals exist and their ranges, so the form never keeps its own copy.
2. On submit, `validate()` checks the numbers in the browser; then `submitVitals()` sends `POST /api/v1/health/vitals`.
3. `submit_vitals` checks the citizen allows health tracking, then calls `record_reading()`.
4. `record_reading()` loads up to 120 earlier readings, saves the new one, calls `score_reading()`, saves the score, and writes an alert if flagged.
5. The route answers with the result: flagged or not, severity, the reason, and a per-vital breakdown (`checks`). The page shows it.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/services/metrics.py` · validate_vitals

Called when the request body is read: anything physically impossible is a typo, not an anomaly.

```
def validate_vitals(vitals: dict[str, float | None]) -> dict[str, float]:
    """Drop unknown keys and blanks; reject physically implausible values as input errors."""
    clean: dict[str, float] = {}
    for key, value in vitals.items():
        spec = VITALS.get(key)
        if spec is None or value is None:
            continue
        low, high = spec.plausible
        if not low <= float(value) <= high:
            raise ImplausibleVitalError(
                f"{spec.label} {value:g} {spec.unit} is not a plausible reading "
                f"(expected {low:g}–{high:g}). Check for a typo."
            )
        clean[key] = float(value)
    return clean
```

What the code means

- `dict[str, float | None]` is a dictionary from vital name to number (or nothing, for an empty field).
- `VITALS.get(key)` looks the vital up; unknown names and blanks are skipped with `continue`.
- `low, high = spec.plausible` unpacks a pair. `low <= float(value) <= high` is a chained comparison: both must hold.
- `raise ImplausibleVitalError(...)` stops processing; the API turns it into a 422 error with the message, for example "Heart rate 900 bpm is not a plausible reading".
- `{value:g}` in the f-string prints a number without unnecessary decimals.

2. `backend/app/models/health_record.py` · HealthRecord

The table that stores one reading.

```
class HealthRecord(UUIDPk, CreatedAt, Base):
    """One submitted reading. Vital keys, units and ranges: services/vitals.py."""

    __tablename__ = "health_records"

    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id"), index=True)
    vitals: Mapped[dict[str, Any]] = mapped_column(JSON)
    symptoms: Mapped[list[str]] = mapped_column(JSON)
    geo_lat: Mapped[float | None]
    geo_lng: Mapped[float | None]
    timestamp: Mapped[datetime]  # when it was measured; created_at is when it was stored

    processed: Mapped["ProcessedData | None"] = relationship(back_populates="health_record", uselist=False)
```

What the code means

- `class HealthRecord(UUIDPk, CreatedAt, Base)` inherits an `id` column and a `created_at` column from small mixin classes (guide 14).
- `Mapped[dict[str, Any]] = mapped_column(JSON)`: the vitals are stored as one JSON object such as {"systolic": 120, "heart_rate": 72}, so adding a new vital needs no table change.
- `ForeignKey("users.id")` links the reading to the citizen; `index=True` makes "all readings of this user" fast.
- `timestamp` is when it was **measured**; `created_at` is when it was **stored** (a citizen can log an earlier measurement).
- `relationship(..., uselist=False)` lets code reach the reading's score as `record.processed`.

3. `backend/app/api/routes/vitals.py` · submit_vitals

```python
@router.post("/vitals", response_model=VitalsResult, status_code=status.HTTP_201_CREATED)
async def submit_vitals(
    data: VitalsIn, user: User = Depends(require_onboarding_user), db: AsyncSession = Depends(get_session)
) -> VitalsResult:
    if not user.consent:
        raise HTTPException(status.HTTP_403_FORBIDDEN, TRACKING_OFF)
    measured_at = data.measured_at or now_utc()
    recorded = await record_reading(db, user, data.vitals, data.symptoms, measured_at)
    await db.commit()

    score = recorded.score
    return VitalsResult(
        id=recorded.record.id,
        measured_at=measured_at,
        flagged=score.is_anomaly,
        anomaly_score=score.anomaly_score,
        severity=severity_of(score.anomaly_score) if score.is_anomaly else None,
        driver=top_signal(score.features) if score.is_anomaly else None,
        signal=recorded.signal,
        features=score.features,
        basis=score.basis,
        earlier_readings=recorded.earlier_readings,
        basis_note=_basis_note(score),
        reviewed_by_someone=recorded.reviewed_by_someone,
        model_version=score.model_version,
        checks=_checks(score),
        watch=score.watch,
    )
```

What the code means

- `@router.post("/vitals", response_model=VitalsResult, status_code=201)` registers `POST /api/v1/health/vitals`; the answer is shaped by `VitalsResult` and has status 201 (created).
- `data: VitalsIn` is the validated JSON body. `Depends(require_onboarding_user)` guarantees a verified citizen.
- `if not user.consent: raise HTTPException(403, ...)`: if health tracking is turned off in Settings, nothing is saved.
- `data.measured_at or now_utc()`: use the time the citizen gave, otherwise now.
- `await record_reading(...)` does the real work; `await db.commit()` saves it all in one transaction.
- The `VitalsResult(...)` fields come from the score: `severity_of(...) if score.is_anomaly else None` only fills the severity for flagged readings.

4. `backend/app/services/readings.py` · record_reading

```python
async def record_reading(
    session: AsyncSession,
    citizen: User,
    vitals: dict[str, float],
    symptoms: list[str],
    measured_at: datetime,
    *,
    stored_at: datetime | None = None,
) -> Recorded:
    """Store and score one already-validated reading. Flushes; the caller commits."""
    history = await earlier_vitals(session, citizen.id, measured_at)
    record = HealthRecord(
        user_id=citizen.id,
        vitals=vitals,
        symptoms=symptoms,
        geo_lat=citizen.geo_lat,
        geo_lng=citizen.geo_lng,
        timestamp=measured_at,
    )
    if stored_at is not None:
        record.created_at = stored_at
    session.add(record)
    await session.flush()

    score = score_reading(citizen.id, vitals, history)
    processed = ProcessedData(
```

```
        health_record_id=record.id,
        user_id=citizen.id,
        features=score.features,
        anomaly_score=score.anomaly_score,
        is_anomaly=score.is_anomaly,
        basis=score.basis,
        model_version=score.model_version,
        details=score.details_json(),
    )
    if stored_at is not None:
        processed.processed_at = stored_at
    session.add(processed)

    signal = describe_signal(score.features, vitals) if score.is_anomaly else None
    reviewed = await workers_serving(session, citizen.phc_id) > 0
    if score.is_anomaly:
        alert = Alert(
            user_id=citizen.id,
            alert_type=AlertType.personal,
            severity=alert_severity(severity_of(score.anomaly_score)),
            message=_citizen_message(signal or "one of your values is outside its usual range", measured_at,
                 reviewed),
        )
        if stored_at is not None:
            alert.created_at = stored_at
        session.add(alert)
    await session.flush()
    return Recorded(record, processed, score, len(history), signal, reviewed)
```

What the code means

- The `*` in the parameters means everything after it must be passed by name (`stored_at=...`). Only the seed scripts use `stored_at`.
- `earlier_vitals(...)` loads up to 120 earlier readings, oldest first; these are the person's history for the model.
- `session.add(record)` then `await session.flush()` sends the row to the database so it gets its `id`, without committing yet.
- `score_reading(citizen.id, vitals, history)` is the anomaly model (next items).
- A `ProcessedData` row stores the score plus how it was judged (`basis`, `model_version`, `details`), so every flag can be explained later.
- `workers_serving(...) > 0` checks whether anyone reviews readings at the citizen's centre; the alert text changes accordingly.
- `return Recorded(...)` hands everything back in one small dataclass.

5. `backend/app/services/scoring.py` · score_reading

The entry point of the anomaly model. Each vital is judged on its own by `_judge()` (not shown), then combined.

```
def score_reading(user_id: uuid.UUID, vitals: dict[str, float], history: list[dict[str, float]]) -> ScoreRes
    ult:
    """Returns anomaly_score (0–1), is_anomaly, and features:
    a per-vital contribution dict such as {"systolic": 2.8}.

    `history` is this person's earlier readings, oldest first, excluding this one.
    A vital is scored if it has a published band or a personal baseline.
    """
    del user_id  # everything personal arrives in `history`
    details: dict[str, VitalDetail] = {}
    for key, value in vitals.items():
        spec = VITALS.get(key)
        if spec is None:
            continue
        earlier = [float(h[key]) for h in history if h.get(key) is not None]
        if spec.normal is None and len(earlier) < MIN_BASELINE_READINGS:
            continue  # no band and no baseline yet: nothing to compare with (weight)
        details[key] = _judge(key, float(value), earlier)

    features = {key: round(_contribution(d), 2) for key, d in details.items()}
    peak = max(features.values(), default=0.0)
    return ScoreResult(
        anomaly_score=round(min(1.0, peak / SCORE_CEILING), 3),
```

```
245         is_anomaly=peak > 0,
246         features=features,
247         basis="personal_baseline" if any(d.weight > 0 for d in details.values()) else "published_ranges",
248         details=details,
249         watch=[key for key, d in details.items() if not d.confirmed],
250     )
```

What the code means

- `del user_id`: the parameter is part of the fixed signature, but everything personal arrives in `history`.
- `earlier = [float(h[key]) for h in history if h.get(key) is not None]`: this vital's earlier values, skipping readings that did not include it.
- Weight has no published range, so it is only scored once there is a personal baseline (`continue` otherwise).
- `features` maps each vital to its contribution; `peak` is the largest. The score is `peak / 4`, capped at 1 with `min(1.0, ...)`.
- `basis` says whether any personal baseline was used; `watch` lists vitals that were unusual for the person but still need a second reading to confirm.

6. `backend/app/services/scoring.py` · _contribution

How much one vital adds to the score. The rules of the model in eight lines.

```
213 def _contribution(d: VitalDetail) -> float:
214     """What this vital adds to the score: 0 unless it passes FLAG_THRESHOLD or a safety limit."""
215     if not d.confirmed:
216         return 0.0
217     blended = (1 - d.weight) * d.published + d.weight * (d.personal or 0.0)
218     if d.safety:
219         return max(blended, d.published, FLAG_THRESHOLD)
220     return blended if blended >= FLAG_THRESHOLD else 0.0
```

What the code means

- `d.confirmed` is false for an in-range change that is only unusual for this person and was not repeated: it counts as 0 for now.
- `blended = (1 - d.weight) * d.published + d.weight * d.personal`: a mix of "how far outside the published range" and "how far from your own usual". The weight grows from 0 (new user) to 1 (21+ readings).
- `d.personal or 0.0` uses 0 when there is no personal part yet.
- Past a safety limit (for example fever 38.0 C), the vital always counts, at least `FLAG_THRESHOLD`.
- Otherwise it counts only from `FLAG_THRESHOLD` (0.5) up, so tiny excesses are ignored. The ML Report explains how these numbers were chosen.

7. `backend/app/services/triage.py` · severity_of

```
22 def severity_of(anomaly_score: float) -> Severity:
23     for name, floor in BANDS:
24         if anomaly_score >= floor:
25             return name
26     return "low"
```

What the code means

- `BANDS` is a tuple of (name, lowest score) pairs, worst first: critical 0.75, high 0.5, moderate 0.25, low 0.
- The loop returns the first band the score reaches. The SQL version used by worker pages is built from the same `BANDS`, so they always agree.

8. `frontend/src/pages/citizen/SubmitVitals.tsx` · validate

The same plausibility check, done in the browser first so most typos are caught without a request.

```
function validate(metrics: Metric[], values: Record<string, string>): { vitals: Record<string, number>; errors: Record<string, string> } {
  const vitals: Record<string, number> = {};
  const errors: Record<string, string> = {};
  for (const m of metrics) {
    const raw = values[m.key]?.trim();
    if (!raw) continue;
    const n = Number(raw);
    if (!Number.isFinite(n)) errors[m.key] = "Enter a number";
    else if (n < m.plausible[0] || n > m.plausible[1]) errors[m.key] = `That looks like a typo: expected ${m.plausible[0]}–${m.plausible[1]}`;
    else vitals[m.key] = n;
  }
  if (vitals.systolic !== undefined && vitals.diastolic !== undefined && vitals.diastolic >= vitals.systolic) {
    errors.diastolic = "Diastolic should be lower than systolic. Are they swapped?";
  }
  return { vitals, errors };
}
```

What the code means

- `Record<string, number>` is a TypeScript type for "an object whose keys are strings and values are numbers".
- `values[m.key]?.trim()`: the typed text without spaces; `?.` avoids a crash when the field was never touched.
- `Number.isFinite(n)` rejects text that is not a number.
- The extra check catches swapped blood pressure fields (diastolic must be lower than systolic).
- It returns both the clean numbers and an error per field; the page shows the errors next to the inputs.

9. `frontend/src/services/health.ts` · getMetrics

```
export function getMetrics(): Promise<MetricsInfo> {
  metricsCache ??= api
    .get<MetricsInfo>("/health/metrics")
    .then((r) => r.data)
    .catch((e) => {
      metricsCache = null;
      throw e;
    });
  return metricsCache;
}
```

What the code means

- `metricsCache ??= api.get(...)` only sends the request the first time; later calls reuse the same Promise, because the list of vitals never changes during a session.
- `.then((r) => r.data)` keeps just the JSON body. `.catch(...)` clears the cache on failure so the next call tries again, then re-throws the error with `throw e`.

10. `frontend/src/components/app/ModelInsights.tsx` · verdict

Turns one vital's check into the words in the "Result" column.

```
function verdict(c: VitalCheck, who: Audience): { text: string; cls: string } {
  if (c.safety) return { text: "Past a safety limit", cls: "font-semibold text-sev-high" };
  if (c.contribution > 0) return { text: "Unusual", cls: "font-semibold text-sev-high" };
  if (!c.confirmed) {
    return { text: who === "citizen" ? "Unusual for you, watching the next reading" : "Unusual for them, unconfirmed", cls: "text-warn" };
  }
  return { text: "As expected", cls: "text-success" };
}
```

What the code means

- The order of the `if`s matters: a safety limit wins, then "counted toward the flag", then "waiting for a second reading", otherwise "As expected".

- `who === "citizen"` switches the wording between "you" (citizen screen) and "them" (worker screen); the same component serves both.
- It returns an object with the text and a Tailwind class for its colour.

Syntax used in this feature

Code	What it means
`@router.post(..., status_code=201)`	Register a POST endpoint that answers 201 Created.
`Depends(require_onboarding_user)`	Only a verified citizen may call it.
`mapped_column(JSON)`	A database column that stores a JSON object.
`await session.flush()`	Send pending rows to the database (to get ids) without committing.
`min(1.0, x)`	Cap a number at 1.
`a ??= b`	Assign b only if a is still empty (used for caching).
`x?.trim()`	Call trim only if x exists.

Try it yourself

- Log a heart rate of 900: the browser refuses it (plausible range). Log 125: it is flagged at once (safety limit).
- Change `FLAG_THRESHOLD` in `scoring.py` and run `pytest tests/test_scoring.py` to see which rules change.